fn conservative_continuous_gaussian_tail_to_alpha

Michael Shoemate

April 20, 2026

Proof for `conservative_continuous_gaussian_tail_to_alpha`.

1 Hoare Triple

Precondition

Compiler-verified

- Argument `scale` is of type `f64`
- Argument `tail` is of type `f64`

User-verified

- `scale` > 0
- `tail` > 0

Pseudocode

```
def conservative_continuous_gaussian_tail_to_alpha(scale: f64, tail: f64) -> f64:
    # the SQRT_2 constant is already rounded down
    SQRT_2_CEIL = SQRT_2.next_up_()

    t = tail.neg_inf_div(scale).neg_inf_div(SQRT_2_CEIL)
    # round down to nearest smaller f32
    t = f64(f32.neg_inf_cast(t))
    # erfc error is at most 1 f32 ulp (see erfc_err_analysis.py)
    t = f32.inf_cast(erfc(t)).next_up_()

    return f64(t).inf_div(2.0)
```

Postcondition

Returns `Ok(out)`, where `out` is at least $\Pr[X > \texttt{tail}]$ for $X \sim \mathcal{N}(0, \texttt{scale})$, assuming `tail` > 0, or `Err(e)` if any numerical computation overflows.

2 Proof

Definition 2.1. Define $X \sim \mathcal{N}(0, \texttt{scale})$, where `scale` > 0. Then X follows the continuous gaussian distribution:

$$f(x) = \frac{1}{\texttt{scale}\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x}{\texttt{scale}}\right)^2} \tag{1}$$

Definition 2.2. The error function is defined as:

$$\operatorname{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2} dt \tag{2}$$

Definition 2.3. The complementary error function is defined as:

$$\operatorname{erfc}(z) = 1 - \operatorname{erf}(z) \tag{3}$$

Lemma 2.4. The implementation of `erfc` differs from a conservatively rounded implementation by no greater than one 32-bit float ulp.

Proof. The following code conducts an exhaustive search.

```python
# The result from this check is that the erfc function in statrs errs by at most 1 f32 ulp.

# First disagreement is at 0.5.
# Execution is slowest at inputs around 15, before switching to the next approximating curve.

# To use all CPUs, floats are sharded modulo the number of CPUs (less two).
# It may be necessary to restart this program at a later float to free memory.

import struct
import multiprocessing

# pip install gmpy2
import gmpy2
from opendp._data import erfc

# specifically check max ulp distance from a conservative upper bound
gmpy2.get_context().round = gmpy2.RoundUp


def floatToBits(f):
    s = struct.pack(">f", f)
    return struct.unpack(">l", s)[0]


def bitsToFloat(b):
    s = struct.pack(">l", b)
    return struct.unpack(">f", s)[0]


def worker(offset, step):
    max_err = 0
    print(f"running {offset}")
    # iterate through all 32-bit floats
    for bits in range(floatToBits(0.0), floatToBits(float("inf")), step):
        bits += offset
        if offset == 0 and bits > 0 and (bits // step) % 10_000 == 0:
            prop_done = bits / floatToBits(float("inf"))
            print(
                f"{prop_done:.2%} done, with max discovered f32 ulp error of: {max_err}. Currently at: {bitsToFloat(bits)}"
            )

        f32 = bitsToFloat(bits)
        f32_ulp_err = abs(floatToBits(erfc(f32)) - floatToBits(gmpy2.erfc(f32)))
        max_err = max(max_err, f32_ulp_err)
    print(max_err)


if __name__ == "__main__":
```

```python
    n_cpus = multiprocessing.cpu_count() - 2
    processes = []
    for cpu in range(n_cpus):
        p = multiprocessing.Process(target=worker, args=(cpu, n_cpus))
        p.start()
        processes.append(p)

    [p.join() for p in processes]
```

Upon completion, the greatest discovered error is at most 1 ulp. □

Theorem 2.5. Assume $X \sim \mathcal{N}(0, \texttt{scale})$ and $\texttt{tail} > 0$. Then

$$\alpha = P[X > \texttt{tail}] = \frac{1}{2}\operatorname{erfc}\left(\frac{\texttt{tail}}{\texttt{scale}\sqrt{2}}\right) \tag{4}$$

Proof.

$$\begin{aligned}
\alpha &= P[X > \texttt{tail}] \\
&= P[X \geq \texttt{tail}] && \text{because } X \text{ is continuous} \\
&= \int_{\texttt{tail}}^{\infty} \frac{1}{\texttt{scale}\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x}{\texttt{scale}}\right)^2} dx && \text{by 2.1} \\
&= \frac{1}{\sqrt{\pi}} \int_{\texttt{tail}/(\texttt{scale}\sqrt{2})}^{\infty} e^{-u^2} du && \text{with } u = \frac{x}{\texttt{scale}\sqrt{2}} \\
&= \frac{1}{2}\left(1 - \operatorname{erf}\frac{\texttt{tail}}{\texttt{scale}\sqrt{2}}\right) && \text{by 2.2} \\
&= \frac{1}{2}\operatorname{erfc}\left(\frac{\texttt{tail}}{\texttt{scale}\sqrt{2}}\right) && \text{by 2.3}
\end{aligned}$$

The implementation computes $\frac{\texttt{tail}}{\texttt{scale}\sqrt{2}}$ with conservative downward rounding, so the input to `erfc` is no greater than the exact value. Since `erfc` is monotonically decreasing, this yields a value no smaller than the exact tail probability expression above. The outcome of `erfc` is then increased by one 32-bit float ulp, which compensates for implementation error by 2.4. Therefore the returned result is at least $\Pr[X > \texttt{tail}]$. □